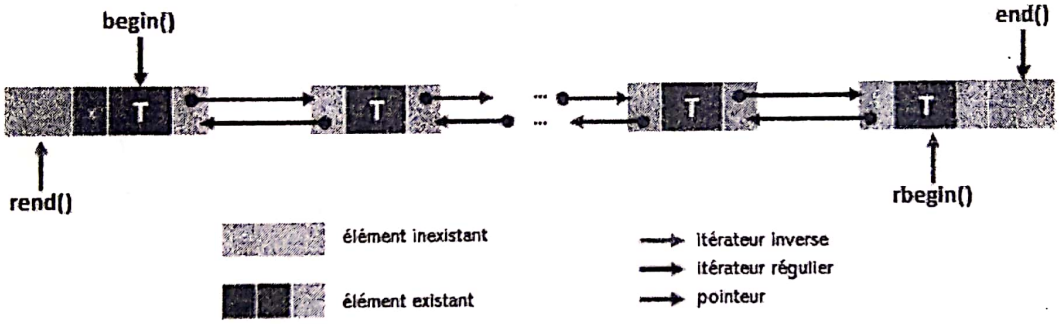
La classe list, définie dans le fichier d'en-tête <list>, est un conteneur de séquence avec une insertion et une suppression rapide à tout moment. Cela signifie que nous pouvons insérer ou supprimer facilement à tout moment dans une liste.

Les listes C++ peuvent être réduites ou étendues des deux côtés au moment de l'exécution. Le besoin de stockage est automatiquement satisfait par l'allocateur interne.

Une liste ne prend pas en charge l'accès aléatoire pour récupérer ou modifier la valeur d'un élément à l'aide de l'opérateur d'index ou de la fonction membre at() car la liste est implémentée comme une liste doublement chaînée, pas comme un tableau dans la mémoire tas. Si nous voulons accéder à la liste de façon aléatoire, nous devons utiliser un itérateur qui se déplace jusqu'à l'élément désiré et y accède ensuite.

La figure ci-dessous décrit une liste et montre que chaque nœud possède une section de données et deux pointeurs, l'un pointant vers le nœud précédent et l'autre pointant vers le nœud suivant.



Les conteneurs de liste sont implémentés comme des listes doublement chainées. Les listes doublement chainées peuvent stocker chacun des éléments qu'elles contiennent dans des emplacements de stockage différents et non liés. L'ordre est maintenu en interne par l'association à chaque élément d'un lien (pointeur) vers l'élément qui le précède et d'un lien vers l'élément qui le suit.

Par rapport à d'autres conteneurs de séquence standard de base (array, vector et deque), les listes fonctionnent généralement mieux pour insérer, extraire et déplacer des éléments dans n'importe quelle position du conteneur pour lequel un itérateur a déjà été obtenu, et donc également dans les algorithmes qui font un usage intensif d'entre eux, comme les algorithmes de tri.

```
#include<iostream>
#include<list>
using namespace std;
int main (){
   list<int> l; // créer une liste ( d'entiers) vide l
   return 0;
}
```

Exercice1: insertion

Question 1: Donner les résultats des programmes suivants:

#include<iostream>
#include<list>
using namespace std;
int main (){
list<int> l;
list<int>::iterator it;
it=l.begin();
l.insert(it,3);
l.insert(it,13);
l.insert(it,10);
for (it = l.begin(); it != l.end(); ++it)
cout << *it << endl;
return 0;
}
	#include<iostream>
	#include<list>
	using namespace std;
	int main (){
	list<int> l;
	list<int>::iterator it;
	it=l.begin();
	l.insert(it,3);
	l.insert(it++,13);
	l.insert(it,10);
	for (it = l.begin(); it != l.end(); ++it)
	cout << *it << endl;
	return 0;
	}

3 13 10	10 3 13

Question 2: Donner les résultats des programmes suivants:

#include<iostream>
#include<list>
using namespace std;
int main (){
list<int> l;
list<int>::iterator it;
l.push_front(10);
l.push_front(20);
l.push_front(30);
for (it = l.begin(); it != l.end(); ++it)
cout << *it << endl;
return 0;
}
	#include<iostream>
	#include<list>
	using namespace std;
	int main (){
	list<int> l;
	list<int>::iterator it;
	l.push_back(10);
	l.push_back (20);
	l.push_back (30);
	for (it = l.begin(); it != l.end(); ++it)
	cout << *it << endl;
	return 0;
	}

30 20 10	10 20 30

Exercice2: size

Question 1: Donner les résultats des programmes suivants:

#include<iostream> #include<list> using namespace std; int main (){	#include<iostream> #include<list> using namespace std; int main (){

```
list<int> l;
list<int>::iterator it;
l.push_front(10);
l.push_front(20);
l.push_front(30);
cout<<l.size()<<endl;
cout<<l.max_size()<<endl;
cout<<l.empty()<<endl;
return 0;
}
```

3
la taille maximale q'une liste peut avoir
10

```
list<int> l;
list<int>::iterator it;
l.push_front(10);
l.push_front(20);
l.push_front(30);
cout<<l.front()<<endl;
cout<<l.back()<<endl;
return 0;
}
```

30
20
10.

Question 2: suppression Donner les résultats des programmes suivants:

```
#include<iostream>
#include<list>
using namespace std;
int main (){
list<int> l;
list<int>::iterator it;
l.push_front(10);
l.push_front(20);
l.push_front(30);
l.push_back(1);
l.push_back(2);
l.push_back(3);
for (it = l.begin(); it != l.end(); ++it)
cout << *it <<" ";
cout<<"\n";
l.pop_back(); Supprimer le dernier élu
for (it = l.begin(); it != l.end(); ++it)
cout << *it <<" ";
cout<<"\n";
return 0;
}
```

30 20 10 1 2 3
20
10 30 20 10 1 2

```
#include<iostream>
#include<list>
using namespace std;
int main (){
list<int> l;
list<int>::iterator it;
l.push_front(10);
l.push_front(20);
l.push_front(30);
l.push_back(1);
l.push_back(2);
l.push_back(3);
for (it = l.begin(); it != l.end(); ++it)
cout << *it <<" ";
cout<<"\n";
l.pop_front();
for (it = l.begin(); it != l.end(); ++it)
cout << *it <<" ";
cout<<"\n";
return 0;
}
```

30 20 10 1 2 3
20 10 1 2 3

```
#include<iostream>
#include<list>
using namespace std;
```

```
#include<iostream>
#include<list>
using namespace std;
```

```
int main (){
   list<int> l;
   list<int>::iterator it;
   l.push_front(10);
   l.push_front(20);
   l.push_front(30);
   l.push_back(1);
   l.push_back(2);
   l.push_back(3);
    for (it = l.begin(); it != l.end(); ++it)
        cout << *it <<" ";
    cout<<"\n";

    it=l.begin();
   it=++it;
    l.erase(it);
    for (it = l.begin(); it != l.end(); ++it)
        cout << *it <<" ";
    cout<<"\n";
    return 0;
}
```

30 20 10 1 2 3
30 10 1 2 3

```
int main (){
   list<int> l;
   list<int>::iterator it;
   l.push_front(10);
   l.push_front(20);
   l.push_front(30);
   l.push_back(1);
   l.push_back(2);
   l.push_back(3);
   for (it = l.begin(); it != l.end(); ++it)
        cout << *it <<" ";
    cout<<"\n";
    l.clear();
    for (it = l.begin(); it != l.end(); ++it)
        cout << *it <<" ";
     cout<<"\n";
     return 0;
}
```

30 20 10 1 2 3
Rien (vide)

Exercice2: sort remove

```
#include<iostream>
#include<list>
using namespace std;
int main (){
   list<int> l;
   list<int>::iterator it;
   l.push_front(10);
   l.push_front(20);
   l.push_front(30);
   l.push_back(1);
   l.push_back(2);
   l.push_back(3);
  for (it = l.begin(); it != l.end(); ++it)
      cout << *it <<" ";
  cout<<"\n";
  l.sort();
  for (it = l.begin(); it != l.end(); ++it)
      cout << *it <<" ";
```

permet de trier la list.

```
#include<iostream>
#include<list>
using namespace std;
int main (){
   list<int> l;
   list<int>::iterator it;
   l.push_front(10);
   l.push_front(20);
   l.push_front(30);
   l.push_back(1);
   l.push_back(2);
   l.push_back(3);
   for (it = l.begin(); it != l.end(); ++it)
       cout << *it <<" ";
   cout<<"\n";
   l.remove(10);
   for (it = l.begin(); it != l.end(); ++it)
       cout << *it <<" ";
```

permet sup tous les répétitions de 10

30 20 10 1 2 3
1 2 3 10 20 30

30 20 10 1 2 3
30 20 1 2 3

cout<<"\n"; return 0; }	cout<<"\n"; return 0; }

Exemple Complet

On considère le programme suivant:

Complexe.h	Complexe.cpp

#include <iostream>
 	
class Complexe
{
private:
float Re;
float Im;
public:
Complexe(float rr=0,float ii=0);
void afficherComplexe();
 	
};
	#include <iostream>
	#include "Complexe.h"
	using namespace std;
	
	Complexe::Complexe(float rr,float ii){ this->Re=rr;this->Im=ii;};
	void Complexe::afficherComplexe(){cout<<this->Re<<"+i*"<<this->Im<<endl; };

main.cpp

```
#include <iostream>
#include<list>
#include "Complexe.h"
using namespace std;
int main(int argc, char** argv) {

        list<Complexe> L;
  list<Complexe>::iterator it;
  Complexe Z1(2,4),Z2,Z3(1,2);
  L.push_front(Z1);
  L.push_front(Z2);
  L.push_back(Z3);

        for (it = L.begin(); it != L.end(); ++it)
            it->afficherComplexe();
        cout<<"\n";
  return 0;
}
```